INTERNSHIP TASK REPORT

API Data Processing Script

Week-02 | Task 2 | Khansa Malik

Task Objective	Build a Python script that consumes JSON data, extracts and cleans selected fields, calculates summary statistics, and exports the processed results.
Technology	Python 3.7+ using the standard library only; JSONPlaceholder public API with a local sample-data fallback.

1. Approach and Implementation

The script retrieves user records from **https://jsonplaceholder.typicode.com/users**, then processes the response through a simple validation and transformation pipeline.

- Data acquisition: requests are handled with timeout and response-status checks. When the API is unavailable, the script automatically uses the bundled sample_data.json fallback.
- Field extraction and cleaning: name, username, email, city, and company are selected; whitespace is stripped, emails are normalized to lowercase, and records missing a name or email are skipped.
- Analysis and export: the cleaned dataset is summarized using record count, average/minimum/maximum name length, unique city count, and unique company count, then written to output_data.csv and summary_stats.txt.

2. Error Handling

The implementation includes checks for network failures, request timeouts, non-200 responses, malformed JSON, malformed records, and file-write errors. This prevents a single bad input or unavailable API from terminating the workflow without a useful fallback.

3. Testing and Final Result

- The script was designed to run directly with: python3 process_api_data.py
- Successful processing produces two final artifacts: output_data.csv (cleaned records) and summary_stats.txt (calculated statistics).
- Fallback behavior was included so the same processing flow can complete using sample_data.json when live API access is unavailable.

Output	Purpose
output_data.csv	Cleaned and selected API records
summary_stats.txt	Summary statistics for the processed dataset

4. Key Lessons and Next Iteration

This task reinforced practical API consumption, defensive programming, data cleaning, validation, and file-based reporting. The next iteration could add structured logging, configurable input/output paths, automated unit tests, schema validation, larger datasets, and more advanced statistics while preserving the fallback mechanism.